

PostgreSQL Replication Slot

Incident Response Runbook

PostgreSQL 17 on AlmaLinux 9

Field	Value
Document Version	1.0
Last Updated	March 19, 2026
Author	DBA Operations Team
Classification	Internal / Confidential
Replication Slot Name	ssncpri
Data Directory (PGDATA)	/apps/pgsql_data/17
WAL Directory	/apps/pgsql_data/17/pg_wal
PostgreSQL Log Directory	/apps/logs
postgresql.conf	/apps/pgsql_data/17/postgresql.conf
pg_hba.conf	/apps/pgsql_data/17/pg_hba.conf
postgresql.auto.conf	/apps/pgsql_data/17/postgresql.auto.conf
Service Name	postgresql-17
PostgreSQL Version	17
Operating System	AlmaLinux 9

WARNING: Test ALL procedures in a non-production environment before executing on production systems.

1. Overview & Architecture

This runbook provides detailed diagnostic and recovery procedures for every known scenario that causes the replication slot 'ssncpri' to become inactive or invalidated on the Primary server, halting WAL replication to the Standby.

1.1 Environment Architecture

Component	Primary Server	Standby Server
Role	Read-Write (WAL source)	Read-Only (WAL consumer)
Key Process	walsender	walreceiver
Replication Slot	'ssncpri' — created & owned here	Connects using primary_slot_name='ssncpri'
Data Directory	/apps/pgsql_data/17	/apps/pgsql_data/17
pg_wal Location	/apps/pgsql_data/17/pg_wal	/apps/pgsql_data/17/pg_wal
Log Directory	/apps/logs	/apps/logs
Config File	/apps/pgsql_data/17/postgresql.conf	/apps/pgsql_data/17/postgresql.conf
pg_hba.conf	/apps/pgsql_data/17/pg_hba.conf — must allow replication	N/A
postgresql.auto.conf	/apps/pgsql_data/17/postgresql.auto.conf	/apps/pgsql_data/17/postgresql.auto.conf — contains primary_conninfo
Service	systemctl * postgresql-17	systemctl * postgresql-17

1.2 Key File Paths Quick Reference

Item	Path
PGDATA	/apps/pgsql_data/17
pg_wal	/apps/pgsql_data/17/pg_wal
PostgreSQL logs	/apps/logs/postgresql-*.log
postgresql.conf	/apps/pgsql_data/17/postgresql.conf
pg_hba.conf	/apps/pgsql_data/17/pg_hba.conf
postgresql.auto.conf	/apps/pgsql_data/17/postgresql.auto.conf
Monitoring script	/opt/scripts/pg_replication_slot_monitor.sh
Monitor log	/apps/logs/pg_replication_slot_monitor.log

Item	Path
Monitoring config	/opt/scripts/monitor.conf

2. First-Response Diagnostics

When a replication issue is suspected, run ALL of the following diagnostic steps on the PRIMARY server BEFORE taking any corrective action. Copy-paste the exact commands below. Document every output for the incident ticket.

Step 1: Verify you are on the PRIMARY server

1. Confirm server role:

```
psql -U postgres -d postgres -c "SELECT pg_is_in_recovery();"
```

Expected output: 'f' (false) = this is the Primary. If 't', you are on the Standby.

Step 2: Check the replication slot 'ssncpri'

1. Query full slot status:

```
psql -U postgres -d postgres -c "  
    SELECT slot_name, slot_type, active, active_pid, restart_lsn,  
           wal_status, safe_wal_size, conflicting, invalidation_reason,  
           pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(),  
restart_lsn)) AS lag  
    FROM pg_replication_slots WHERE slot_name = 'ssncpri';  
"
```

2. Record the values of: active, wal_status, conflicting, invalidation_reason, lag.
3. If NO rows returned, the slot has been dropped. Go to Section 3.5.

Step 3: Check streaming replication connections

1. Query pg_stat_replication:

```
psql -U postgres -d postgres -c "  
    SELECT pid, client_addr, application_name, state, sync_state,  
           sent_lsn, write_lsn, flush_lsn, replay_lsn,  
           pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(),  
replay_lsn)) AS replay_lag,  
           EXTRACT(EPOCH FROM replay_lag)::int AS lag_seconds,  
           EXTRACT(EPOCH FROM write_lag)::int AS write_lag_seconds,  
           EXTRACT(EPOCH FROM flush_lag)::int AS flush_lag_seconds  
    FROM pg_stat_replication ORDER BY client_addr;  
"
```

2. If no rows returned, no standby is connected. Record all output.

Step 4: Check pg_wal directory size

1. Measure WAL directory:

```
du -sh /apps/pgsql_data/17/pg_wal
ls -l /apps/pgsql_data/17/pg_wal/0000* | wc -l
```

2. Check disk space on the PGDATA filesystem:

```
df -h /apps/pgsql_data/17
```

Step 5: Check PostgreSQL logs for errors

1. Review the most recent PostgreSQL log file:

```
ls -lt /apps/logs/postgresql-*.log | head -3
tail -200 $(ls -t /apps/logs/postgresql-*.log | head -1) | \
  grep -iE 'FATAL|PANIC|replication|slot|walsender|walreceiver'
```

Step 6: Check critical parameters

1. Query key replication parameters:

```
psql -U postgres -d postgres -c "
  SELECT name, setting, unit FROM pg_settings
  WHERE name IN
  ('wal_level', 'max_replication_slots', 'max_wal_senders',
   'max_slot_wal_keep_size', 'wal_keep_size', 'wal_sender_timeout',
   'hot_standby_feedback') ORDER BY name;
"
```

Step 7: Check network connectivity to Standby

1. Test network from Primary to Standby:

```
ping -c 4 <standby_ip>
nc -zv <standby_ip> 5432
```

2. Test from Standby to Primary:

```
ssh <standby_ip> 'ping -c 4 <primary_ip>'
ssh <standby_ip> 'nc -zv <primary_ip> 5432'
```

Step 8: Check Standby server status (SSH)

1. SSH to Standby and check PostgreSQL status:

```
ssh <standby_ip>
systemctl status postgresql-17
tail -100 /apps/logs/postgresql-*.log
grep primary_conninfo /apps/pgsql_data/17/postgresql.auto.conf
grep primary_slot_name /apps/pgsql_data/17/postgresql.auto.conf
```

2. Check walreceiver status on Standby:

```
psql -U postgres -d postgres -c "SELECT * FROM pg_stat_wal_receiver;"
```

2.1 Decision Matrix

Diagnostic Result	Root Cause	Go To
active=false, conflicting=false, standby postgresql-17 DOWN	Standby down / crashed	Section 3.1
active=false, standby PG running, ping fails	Network failure	Section 3.2
active=false, standby logs show FATAL auth errors	Auth / config failure	Section 3.3
pg_wal > 10 GB, slot 'ssncpri' active=false	WAL accumulation (disk)	Section 3.4
No rows for 'ssncpri' in pg_replication_slots	Slot dropped	Section 3.5
PG fails to start after parameter change	max_replication_slots reduced	Section 3.6
conflicting=true, invalidation_reason set	Slot invalidated (PG 17)	Section 3.7
active=false suddenly, dmesg shows OOM	WAL sender killed	Section 3.8
All slots inactive after Primary restart	Primary restart / crash	Section 3.9
active=false, standby logs show SSL errors	SSL cert expiry	Section 3.10

3. Scenario Runbooks

3.1 Standby Server Down or Crashed

Symptoms: active=false for slot 'ssncpri'; no standby rows in pg_stat_replication; /apps/pgsql_data/17/pg_wal growing.

Impact: WAL accumulates at /apps/pgsql_data/17/pg_wal. If disk fills, Primary PANIC stops all writes.

Root Causes: Standby postgresql-17 stopped, OS crash, hardware failure, OOM, disk full on standby.

Detailed Diagnosis on Primary

1. Confirm slot 'ssncpri' is inactive:

```
psql -U postgres -d postgres -c "  
    SELECT slot_name, active, active_pid, wal_status,  
           pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(),  
restart_lsn)) AS lag  
    FROM pg_replication_slots WHERE slot_name = 'ssncpri';  
"
```

2. Confirm no walsender:

```
psql -U postgres -d postgres -c "SELECT * FROM pg_stat_replication;"
```

3. Check WAL accumulation:

```
du -sh /apps/pgsql_data/17/pg_wal  
df -h /apps/pgsql_data/17
```

Detailed Diagnosis on Standby (SSH)

1. SSH to standby and check OS:

```
ssh <standby_ip>  
uptime  
who -b  
dmesg -T | tail -50
```

2. Check PostgreSQL service:

```
systemctl status postgresql-17  
systemctl is-active postgresql-17
```

3. Check OOM kills:

```
dmesg -T | grep -i 'oom\|killed process'  
journalctl -k | grep -i oom
```

4. Check disk and memory:

```
df -h /apps/pgsql_data/17  
free -m
```

5. Check standby PostgreSQL logs:

```
tail -100 /apps/logs/postgresql-*.log
```

Recovery Steps**1. Start the Standby PostgreSQL service:**

```
systemctl start postgresql-17
```

2. Verify service started:

```
systemctl status postgresql-17
```

```
tail -30 /apps/logs/postgresql-*.log
```

3. Verify walreceiver on Standby:

```
psql -U postgres -d postgres -c "SELECT status, conninfo, slot_name  
FROM pg_stat_wal_receiver;"
```

4. On Primary — verify slot 'ssncpri' is active:

```
psql -U postgres -d postgres -c "  
  SELECT slot_name, active, active_pid FROM pg_replication_slots WHERE  
  slot_name = 'ssncpri';  
"
```

5. Confirm streaming:

```
psql -U postgres -d postgres -c "SELECT client_addr, state, sync_state  
FROM pg_stat_replication;"
```

6. Monitor pg_wal shrinking:

```
watch -n5 'du -sh /apps/pgsql_data/17/pg_wal && df -h  
/apps/pgsql_data/17'
```

7. Verify lag is decreasing:

```
watch -n5 "psql -U postgres -d postgres -Atc \  
  \"SELECT pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(),  
  replay_lsn)) FROM pg_stat_replication;\""
```

TIP: If standby cannot start due to data corruption, skip to Section 3.7 for full standby rebuild.

WARNING: If disk usage on Primary is above 90% and standby cannot recover quickly, consider dropping the slot (Section 3.4).

3.2 Network Connectivity Failure

Symptoms: active=false for 'ssncpri'; standby postgresql-17 running; ping/nc fails.

Impact: WAL accumulates at /apps/pgsql_data/17/pg_wal.

Root Causes: NIC failure, firewall change, VLAN misconfiguration, DNS failure, cable/switch issue.

Detailed Diagnosis

1. From PRIMARY — test connectivity:

```
ping -c 4 <standby_ip>
nc -zv <standby_ip> 5432
traceroute <standby_ip>
nslookup <standby_hostname>
```

2. From STANDBY — test connectivity:

```
ssh <standby_ip>
ping -c 4 <primary_ip>
nc -zv <primary_ip> 5432
traceroute <primary_ip>
```

3. Check firewall on BOTH servers:

```
firewall-cmd --list-all
firewall-cmd --list-ports
iptables -L -n | grep 5432
```

4. Check network interfaces:

```
ip addr show
ip link show
nmcli connection show
nmcli device status
```

5. Check recent network changes:

```
journalctl -u NetworkManager --since '2 hours ago' --no-pager
dmesg -T | grep -iE 'link|eth|net|bond'
```

Recovery Steps

1. Fix the identified network issue.
2. Wait for walreceiver auto-reconnect (60s default).
3. Verify slot 'ssncpri' active on Primary:

```
psql -U postgres -d postgres -c "SELECT slot_name, active FROM
pg_replication_slots WHERE slot_name = 'ssncpri';"
```

4. If not reconnected after 2 min, restart Standby:

```
ssh <standby_ip> 'systemctl restart postgresql-17'
```

5. Verify streaming:

```
psql -U postgres -d postgres -c "SELECT client_addr, state FROM  
pg_stat_replication;"
```

NOTE: Tune `tcp_keepalives_idle=60`, `tcp_keepalives_interval=10`, `tcp_keepalives_count=6` in `/apps/pgsql_data/17/postgresql.conf` on BOTH servers.

3.3 Authentication / Connection Configuration Failure

Symptoms: active=false; standby logs show FATAL: authentication failed, pg_hba.conf rejects, connection refused.

Root Causes: Changed pg_hba.conf, changed replication password, modified primary_conninfo, expired .pgpass.

Detailed Diagnosis

1. Check Standby logs:

```
ssh <standby_ip>  
tail -200 /apps/logs/postgresql-*.log | grep -iE 'FATAL|  
authentication|pg_hba|connection|denied'
```

2. Verify primary_conninfo on Standby:

```
grep primary_conninfo /apps/pgsql_data/17/postgresql.auto.conf
```

3. Verify primary_slot_name:

```
grep primary_slot_name /apps/pgsql_data/17/postgresql.auto.conf
```

Expected: primary_slot_name = 'ssncpri'

4. On Primary — check pg_hba.conf:

```
grep replication /apps/pgsql_data/17/pg_hba.conf
```

Expected: host replication replicator <standby_ip>/32 scram-sha-256

5. Check listen_addresses:

```
grep listen_addresses /apps/pgsql_data/17/postgresql.conf
```

6. Test connection manually from Standby:

```
psql "host=<primary_ip> port=5432 user=replicator dbname=replication  
sslmode=prefer"
```

7. Check .pgpass on Standby:

```
cat ~/.pgpass  
ls -la ~/.pgpass
```

Recovery Steps

1. Fix pg_hba.conf on Primary if needed:

```
# Add/fix in /apps/pgsql_data/17/pg_hba.conf:  
host      replication  replicator      <standby_ip>/32      scram-sha-256
```

2. Reload Primary (no restart needed):

```
psql -U postgres -d postgres -c "SELECT pg_reload_conf();"
```

3. Update .pgpass on Standby if password changed:

```
echo '<primary_ip>:5432:replication:replicator:<new_password>' >  
~/.pgpass  
chmod 600 ~/.pgpass
```

4. Update primary_conninfo on Standby if needed:

```
psql -U postgres -d postgres -c "ALTER SYSTEM SET primary_conninfo =  
'host=<primary_ip> port=5432 user=replicator password=<password>  
application_name=standby';"
```

5. Reload/restart Standby:

```
psql -U postgres -d postgres -c "SELECT pg_reload_conf();"
# If primary_conninfo changed, restart required:
systemctl restart postgresql-17
```

6. Verify slot 'ssncpri' is active on Primary:

```
psql -U postgres -d postgres -c "SELECT slot_name, active FROM  
pg_replication_slots WHERE slot_name = 'ssncpri';"
```

7. Verify walreceiver on Standby:

```
psql -U postgres -d postgres -c "SELECT status, slot_name FROM  
pg_stat_wal_receiver;"
```

3.4 WAL Accumulation Emergency (Disk Pressure)

Symptoms: /apps/pgsql_data/17/pg_wal consuming excessive space; df shows >80% full; slot 'ssncpri' inactive.

Impact: Disk 100% = PostgreSQL PANIC. All writes stop. P1 incident.

Detailed Diagnosis

1. Assess disk immediately:

```
df -h /apps/pgsql_data/17
du -sh /apps/pgsql_data/17/pg_wal
ls -l /apps/pgsql_data/17/pg_wal/0000* | wc -l
```

2. Identify slot causing retention:

```
psql -U postgres -d postgres -c "
    SELECT slot_name, active, wal_status,
           pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(),
restart_lsn)) AS retained
    FROM pg_replication_slots
    ORDER BY pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn) DESC;
"
```

3. Monitor growth rate:

```
du -sb /apps/pgsql_data/17/pg_wal && sleep 60 && du -sb
/apps/pgsql_data/17/pg_wal
```

Recovery Steps

1. FIRST: Try to reconnect standby:

```
ssh <standby_ip> 'systemctl start postgresql-17'
```

2. Wait 2 min and check:

```
psql -U postgres -d postgres -c "SELECT slot_name, active FROM
pg_replication_slots WHERE slot_name = 'ssncpri';"
```

3. If standby cannot reconnect AND disk > 90%, drop the slot:

-- CAUTION: Standby needs full rebuild after this!

```
psql -U postgres -d postgres -c "SELECT
pg_drop_replication_slot('ssncpri');"
```

4. Force checkpoint:

```
psql -U postgres -d postgres -c "CHECKPOINT;"
```

5. Monitor disk recovery:

```
watch -n5 'du -sh /apps/pgsql_data/17/pg_wal && df -h
/apps/pgsql_data/17'
```

6. Recreate slot and plan rebuild:

```
psql -U postgres -d postgres -c "SELECT
pg_create_physical_replication_slot('ssncpri');"
```

Then follow Section 3.7 for full standby rebuild.

WARNING: Dropping slot 'ssncpri' is DESTRUCTIVE. Standby **MUST** be rebuilt. Only do this in a disk emergency.

3.5 Replication Slot Accidentally Dropped

Symptoms: Standby: FATAL: replication slot "ssncpri" does not exist.

Detailed Diagnosis

1. Confirm slot missing on Primary:

```
psql -U postgres -d postgres -c "SELECT * FROM pg_replication_slots;"
```

2. Check Standby logs:

```
ssh <standby_ip> 'tail -50 /apps/logs/postgresql-*.log | grep -i slot'
```

3. Check if WAL was recycled:

```
ls -lt /apps/pgsql_data/17/pg_wal/0000* | tail -5
```

```
ssh <standby_ip> "psql -U postgres -d postgres -Atc 'SELECT  
pg_last_wal_replay_lsn();'"
```

Recovery Steps

1. Recreate the slot on Primary:

```
psql -U postgres -d postgres -c "SELECT  
pg_create_physical_replication_slot('ssncpri');"
```

2. Verify:

```
psql -U postgres -d postgres -c "SELECT slot_name, slot_type, active  
FROM pg_replication_slots WHERE slot_name = 'ssncpri';"
```

3. Verify primary_slot_name on Standby:

```
ssh <standby_ip> "grep primary_slot_name  
/apps/pgsql_data/17/postgresql.auto.conf"
```

4. Restart Standby:

```
ssh <standby_ip> 'systemctl restart postgresql-17'
```

5. Confirm slot 'ssncpri' active:

```
psql -U postgres -d postgres -c "SELECT slot_name, active FROM  
pg_replication_slots WHERE slot_name = 'ssncpri';"
```

6. Confirm streaming:

```
psql -U postgres -d postgres -c "SELECT client_addr, state FROM  
pg_stat_replication;"
```

WARNING: If required WAL was recycled, standby cannot catch up. Rebuild it (Section 3.7).

3.6 max_replication_slots Reduced Below Active Count

Symptoms: PostgreSQL fails to start. Log: FATAL: too many replication slots.

Detailed Diagnosis

1. Check config:

```
grep max_replication_slots /apps/pgsql_data/17/postgresql.conf
grep max_replication_slots /apps/pgsql_data/17/postgresql.auto.conf
```

2. Check logs:

```
tail -50 /apps/logs/postgresql-*.log | grep -iE 'FATAL|
replication_slots'
```

Recovery Steps

1. Increase max_replication_slots:

```
vi /apps/pgsql_data/17/postgresql.conf
# Set: max_replication_slots = 10
```

2. Restart:

```
systemctl restart postgresql-17
```

3. Verify:

```
systemctl status postgresql-17
psql -U postgres -d postgres -c "SELECT count(*) FROM
pg_replication_slots;"
psql -U postgres -d postgres -c "SHOW max_replication_slots;"
```


3.7 Slot Invalidated (PostgreSQL 17) — Full Standby Rebuild

Symptoms: conflicting=true for 'ssncpri'; invalidation_reason set; wal_status=lost.

Impact: Slot PERMANENTLY UNUSABLE. Standby must be rebuilt.

Detailed Diagnosis

1. Confirm invalidation:

```
psql -U postgres -d postgres -c "
    SELECT slot_name, active, conflicting, invalidation_reason,
    wal_status,
           pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(),
restart_lsn)) AS lag
    FROM pg_replication_slots WHERE slot_name = 'ssncpri';
"
```

2. Understand the reason:

invalidation_reason	Meaning	Why
wal_removed	WAL segments recycled	max_slot_wal_keep_size exceeded while standby disconnected
rows_removed	Rows vacuumed away	Logical slot — vacuum removed needed tuples
wal_level_insufficient	wal_level downgraded	wal_level changed to 'minimal'

3. Check max_slot_wal_keep_size:

```
psql -U postgres -d postgres -c "SHOW max_slot_wal_keep_size;"
```

Recovery Steps: Full Standby Rebuild

WARNING: This DESTROYS all data on the standby. Ensure you have Primary IP, replication credentials, and time for backup.

1. On PRIMARY — drop invalidated slot:

```
psql -U postgres -d postgres -c "SELECT
pg_drop_replication_slot('ssncpri');"
```

2. On PRIMARY — create fresh slot:

```
psql -U postgres -d postgres -c "SELECT
pg_create_physical_replication_slot('ssncpri');"
```

3. Verify:

```
psql -U postgres -d postgres -c "SELECT slot_name, active FROM
pg_replication_slots WHERE slot_name = 'ssncpri';"
```

4. Verify max_wal_senders has room:

```
psql -U postgres -d postgres -c "
```

```
SELECT (SELECT count(*) FROM pg_stat_replication) AS current,
       (SELECT setting::int FROM pg_settings WHERE name =
'max_wal_senders') AS max;
"
```

5. On STANDBY — stop PostgreSQL:

```
ssh <standby_ip>
systemctl stop postgresql-17
```

6. On STANDBY — backup and remove old data:

```
mv /apps/pgsql_data/17 /apps/pgsql_data/17_old_$(date +%Y%m%d_%H%M%S)
```

7. On STANDBY — take fresh base backup:

```
pg_basebackup -h <primary_ip> -p 5432 -U replicator \
-D /apps/pgsql_data/17 \
-S ssncpri \
-X stream \
-P -v -R -C
```

NOTE: -R creates standby.signal + primary_conninfo. -C creates slot if missing. -S uses our slot.

8. Verify standby.signal:

```
ls -la /apps/pgsql_data/17/standby.signal
```

9. Verify primary_conninfo and slot name:

```
grep primary_conninfo /apps/pgsql_data/17/postgresql.auto.conf
grep primary_slot_name /apps/pgsql_data/17/postgresql.auto.conf
```

Ensure: primary_slot_name = 'ssncpri'

10. Set permissions:

```
chown -R postgres:postgres /apps/pgsql_data/17
chmod 700 /apps/pgsql_data/17
```

11. Start Standby:

```
systemctl start postgresql-17
```

12. Verify recovery mode:

```
systemctl status postgresql-17
psql -U postgres -d postgres -c "SELECT pg_is_in_recovery();"

```

13. Check walreceiver:

```
psql -U postgres -d postgres -c "SELECT status, slot_name, sender_host
FROM pg_stat_wal_receiver;"

```

14. On PRIMARY — confirm slot 'ssncpri' active:

```
psql -U postgres -d postgres -c "
SELECT slot_name, active, active_pid, wal_status
FROM pg_replication_slots WHERE slot_name = 'ssncpri';
"
```

15. Confirm streaming:

```
psql -U postgres -d postgres -c "
```

```
SELECT client_addr, state, sync_state,  
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(),  
replay_lsn)) AS lag  
FROM pg_stat_replication;  
"
```

16. Monitor until lag = 0:

```
watch -n5 "psql -U postgres -d postgres -Atc \  
  \"SELECT pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(),  
replay_lsn)) FROM pg_stat_replication;\""
```

17. Remove old backup:

```
rm -rf /apps/pgsql_data/17_old_*
```

3.8 WAL Sender Process Killed (OOM / Manual Kill)

Symptoms: active=false for 'ssncpri' suddenly; dmesg shows OOM/killed.

Detailed Diagnosis

1. Check OOM on Primary:

```
dmesg -T | grep -i 'oom\|killed process'
journalctl -k | grep -i oom
```

2. Check if walsender was killed:

```
dmesg -T | grep -i 'walsender\|postgres'
```

3. Check memory:

```
free -m
ps aux --sort=-%mem | head -20
```

4. Check OOM scores:

```
for pid in $(pgrep -f postgres); do
    echo "PID=$pid OOM_ADJ=$(cat /proc/$pid/oom_score_adj 2>/dev/null)"
done
```

Recovery Steps

1. Wait 60s for auto-reconnect, then check:

```
psql -U postgres -d postgres -c "SELECT slot_name, active FROM
pg_replication_slots WHERE slot_name = 'ssncpri';"
```

2. If not active after 2 min, restart Standby:

```
ssh <standby_ip> 'systemctl restart postgresql-17'
```

3. Protect PostgreSQL from OOM:

```
mkdir -p /etc/systemd/system/postgresql-17.service.d
cat > /etc/systemd/system/postgresql-17.service.d/override.conf <<
'EOF'
[Service]
OOMScoreAdjust=-1000
EOF
systemctl daemon-reload
```

4. Review memory settings:

```
psql -U postgres -d postgres -c "SELECT name, setting, unit FROM
pg_settings WHERE name IN
('shared_buffers', 'work_mem', 'maintenance_work_mem');"
```

3.9 Primary Server Restart or Crash

Symptoms: All slots inactive simultaneously after Primary restart.

Detailed Diagnosis

1. Confirm Primary is online:

```
systemctl status postgresql-17
```

```
psql -U postgres -d postgres -c "SELECT pg_is_in_recovery();" 
```

2. Check clean vs crash:

```
tail -100 /apps/logs/postgresql-*.log | grep -iE 'shutdown|crash|recovery|PANIC'
```

3. Check system logs:

```
last reboot | head -5
```

```
journalctl --since '1 hour ago' | grep -iE 'shutdown|reboot|panic|oom'
```

Recovery Steps

1. Wait 60-90s for walreceiver reconnect.

2. Check slot:

```
psql -U postgres -d postgres -c "SELECT slot_name, active FROM pg_replication_slots WHERE slot_name = 'ssncpri';"
```

3. If not active, restart Standby:

```
ssh <standby_ip> 'systemctl restart postgresql-17'
```

4. If WAL corruption suspected:

```
pg_waldump /apps/pgsql_data/17/pg_wal/$(ls /apps/pgsql_data/17/pg_wal/0000* | tail -1 | xargs basename) 2>&1 | tail -10
```

TIP: If WAL is corrupted, promote Standby and rebuild old Primary as new Standby.

3.10 SSL/TLS Certificate Expiry

Symptoms: active=false; standby logs show SSL errors.

Detailed Diagnosis

1. Check cert expiry on Primary:

```
openssl x509 -in /apps/pgsql_data/17/server.crt -noout -dates  
openssl x509 -in /apps/pgsql_data/17/server.crt -noout -subject -  
issuer
```

2. Check ssl enabled:

```
psql -U postgres -d postgres -c "SHOW ssl;"
```

3. Check Standby SSL errors:

```
ssh <standby_ip> 'tail -50 /apps/logs/postgresql-*.log | grep -i ssl'
```

Recovery Steps

1. Replace expired certificates on Primary.

2. Reload Primary (no restart needed for certs):

```
psql -U postgres -d postgres -c "SELECT pg_reload_conf();" 
```

3. If CA changed, update on Standby and restart:

```
ssh <standby_ip> 'systemctl restart postgresql-17'
```

4. Verify slot active:

```
psql -U postgres -d postgres -c "SELECT slot_name, active FROM  
pg_replication_slots WHERE slot_name = 'ssncpri';"
```

4. Preventive Measures

4.1 Recommended Parameter Settings

Parameter	Recommended	Set In	Purpose
max_slot_wal_keep_size	50GB	/apps/pgsql_data/17/postgresql.conf	Prevents infinite WAL growth
wal_keep_size	2GB	/apps/pgsql_data/17/postgresql.conf	Backup WAL retention
max_replication_slots	10	/apps/pgsql_data/17/postgresql.conf	Headroom for slots
max_wal_senders	10	/apps/pgsql_data/17/postgresql.conf	Headroom for connections
wal_sender_timeout	60s	/apps/pgsql_data/17/postgresql.conf	Detect dead connections
wal_receiver_timeout	60s	Standby /apps/pgsql_data/17/postgresql.conf	Detect dead on standby
tcp_keepalives_idle	60	/apps/pgsql_data/17/postgresql.conf	Faster failure detection
tcp_keepalives_interval	10	/apps/pgsql_data/17/postgresql.conf	TCP retry interval
tcp_keepalives_count	6	/apps/pgsql_data/17/postgresql.conf	Max retries
hot_standby_feedback	on	Standby /apps/pgsql_data/17/postgresql.conf	Prevents query conflicts

4.2 Monitoring Script Deployment

1. Copy monitoring script:

```
scp pg_replication_slot_monitor.sh primary:/opt/scripts/
chmod +x /opt/scripts/pg_replication_slot_monitor.sh
```

2. Create config:

```
cat > /opt/scripts/monitor.conf << 'EOF'
PGHOST="localhost"
PGPORT="5432"
PGUSER="postgres"
PGDATABASE="postgres"
ALERT_EMAIL="dba-team@example.com"
```

```
ENABLE_EMAIL_ALERTS="true"
WAL_LAG_WARNING_MB="1024"
WAL_LAG_CRITICAL_MB="5120"
EOF
```

3. Schedule via cron:

```
# /etc/cron.d/pg_repl_monitor
*/5 * * * * postgres /opt/scripts/pg_replication_slot_monitor.sh \
    --config /opt/scripts/monitor.conf >>
    /apps/logs/pg_repl_monitor.log 2>&1
```

4. Test:

```
sudo -u postgres /opt/scripts/pg_replication_slot_monitor.sh --
config /opt/scripts/monitor.conf
```


5. Appendix

5.1 Key Views Reference

View / Function	Run On	Purpose
pg_replication_slots	Primary	Slot metadata, activity, WAL status, invalidation
pg_stat_replication	Primary	Per-standby connection, LSN, lag
pg_stat_wal_receiver	Standby	WAL receiver status, sender info
pg_current_wal_lsn()	Primary	Current WAL write position
pg_last_wal_replay_lsn()	Standby	Last replayed WAL position
pg_wal_lsn_diff(a, b)	Any	Byte difference between LSNs
pg_is_in_recovery()	Any	True if standby
pg_create_physical_replication_slot('ssncpri')	Primary	Create physical slot
pg_drop_replication_slot('ssncpri')	Primary	Drop slot

5.2 WAL Status Lifecycle

wal_status	Meaning	Risk	Action
reserved	WAL within wal_keep_size	None	No action
extended	WAL beyond wal_keep_size, slot prevents removal	Low	Monitor
unreserved	WAL may be removed at next checkpoint	HIGH	Reconnect standby ASAP
lost	Required WAL removed	CRITICAL	Rebuild standby

5.3 Emergency Quick Commands

Action	Command
Check slot 'ssncpri'	SELECT active, wal_status, conflicting FROM pg_replication_slots WHERE slot_name = 'ssncpri';
Check lag	SELECT client_addr, pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), replay_lsn)) FROM pg_stat_replication;
pg_wal size	du -sh /apps/pgsql_data/17/pg_wal

Action	Command
Disk	<code>df -h /apps/pgsql_data/17</code>
Drop slot (EMERGENCY)	<code>SELECT pg_drop_replication_slot('ssncpri');</code>
Recreate slot	<code>SELECT pg_create_physical_replication_slot('ssncpri');</code>
Force checkpoint	<code>CHECKPOINT;</code>
Start standby	<code>systemctl start postgresql-17</code>
Reload config	<code>SELECT pg_reload_conf();</code>

5.4 Emergency Contacts

Role	Contact	Level
On-Call DBA	<update>	L1 — First responder
DBA Team Lead	<update>	L2 — Rebuild decisions
Infrastructure	<update>	Network / firewall
Management	<update>	L3 — Data loss risk

--- End of Runbook ---